Proyecto De Software Basado En Herramientas De Integración Continua

Entrega Proyecto 1 - Escenario 3

Alejandro Montoya Sandoval

Edinson Hernandez

Jonathan Franco Santana

Politécnico Grancolombiano

Integración Continua

John Olarte

10 de noviembre de 2026

Tabla de contenido

Introducción....1

Objetivo General....1

Objetivos específicos....2

Descripción Del Proyecto....2

Arquitectura De La Solución....3

Evidencias De Ejecución....5

Evidencias de levantamiento del entorno....5

Evidencias de arranque de los servicios....6

Comunicación entre los contenedores....6

Prueba de comunicación con la base de datos....8

Evidencias de integración continua....9

Enlace A Repositorio De GitHub....9

Referencias....10

Introducción

La integración continua es una práctica de la ingeniería de software que consiste en integrar el trabajo de todos los miembros de un equipo en un repositorio compartido de forma frecuente, verificando cada integración mediante construcciones y pruebas automatizadas. Su propósito es detectar los errores lo antes posible, reducir el riesgo de las entregas y mantener el software en un estado siempre desplegable (Fowler, 2006).

Para que esa práctica sea viable, el software debe ejecutarse de la misma manera en el equipo de cada desarrollador, en el servidor de integración y en el entorno de producción. La contenerización resuelve ese problema, pues Docker empaqueta la aplicación junto con su sistema operativo base, sus dependencias y su configuración en una unidad portable e inmutable (Docker Inc., 2026). De esta manera se elimina una de las causas más frecuentes de fallo en los procesos automatizados: la diferencia entre entornos.

El presente documento corresponde a la primera entrega del proyecto del módulo, desarrollada en el Escenario 3. De acuerdo con la guía del proyecto, en este punto la solución debe encontrarse alojada en un repositorio de GitHub y debe emplearse Docker para construir dos contenedores que estén comunicados entre sí. A lo largo del documento se describe el proyecto de software seleccionado, la arquitectura de los dos contenedores, el mecanismo mediante el cual se comunican y las evidencias obtenidas durante la ejecución real del entorno.

Objetivo General

Implementar una infraestructura de software basada en contenedores Docker que permita ejecutar una aplicación web y un motor de base de datos como servicios independientes, establecer la comunicación entre ambos y alojar la solución en un repositorio de GitHub.

Objetivos específicos

- Configurar un repositorio GitHub para el control de versiones del proyecto.
- Construir la imagen Docker de la aplicación principal a partir de un Dockerfile propio.
- Implementar un segundo servicio de base de datos dentro de un contenedor independiente.
- Establecer la comunicación entre los contenedores mediante una red definida por el usuario.
- Orquestar el ciclo de vida de ambos servicios utilizando Docker Compose.
- Aplicar buenas prácticas de seguridad y gestión de configuraciones.
- Automatizar procesos básicos mediante un pipeline de integración continua.
- Documentar y evidenciar el correcto funcionamiento de la solución desarrollada.

Descripción Del Proyecto

Gestor de Tareas es un proyecto académico que implementa una interfaz de programación de aplicaciones (API) de tipo REST para registrar, consultar y eliminar las tareas de un equipo de desarrollo. La aplicación está escrita en Python con el framework Flask y se sirve mediante Gunicorn, mientras que la información se almacena de forma persistente en PostgreSQL 16 (Pallets, 2026; PostgreSQL Global Development Group, 2026).

La infraestructura se implementa mediante dos contenedores Docker independientes, administrados con Docker Compose y conectados a través de una red privada. El caso de negocio se mantuvo deliberadamente acotado para que el foco del trabajo permanezca en las herramientas de integración continua y no en la lógica de la aplicación; aun así, el proyecto exige de forma natural dos componentes separados, que es justamente lo que la entrega requiere.

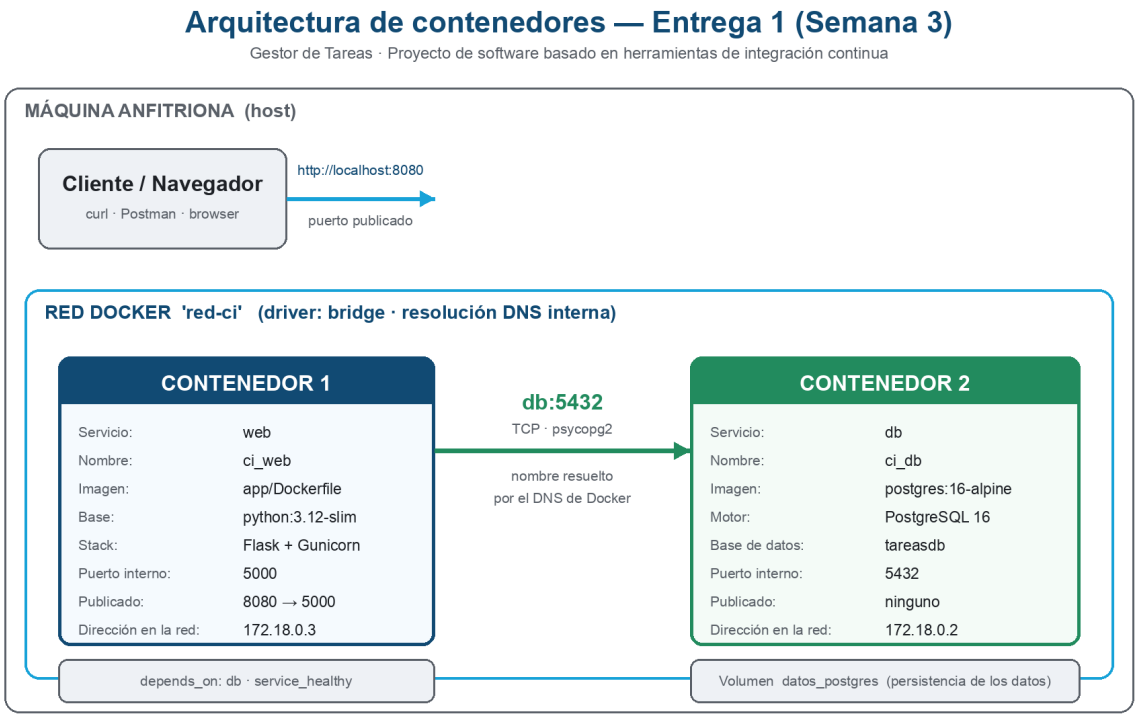
El proyecto evolucionará hacia un pipeline de integración y entrega continua que permitirá automatizar validaciones, pruebas y despliegues con Jenkins y Travis CI en las entregas siguientes. El control de versiones se encuentra alojado en un repositorio de GitHub con una estrategia de ramas compuesta por main, develop y ramas de funcionalidad.

Arquitectura De La Solución

La plataforma está compuesta por dos contenedores que se ejecutan de forma aislada y que se comunican exclusivamente a través de una red interna de Docker creada para el proyecto. El primer contenedor, denominado ci_web, ejecuta el servicio de aplicación y es el único que publica un puerto hacia la máquina anfitriona. El segundo, denominado ci_db, ejecuta el motor de base de datos y no publica ningún puerto, por lo que solo es accesible desde dentro de la red del proyecto.

Figura 1

Arquitectura de los dos contenedores del proyecto.



Nota. Elaboración propia.

La comunicación entre ambos contenedores se apoya en la resolución de nombres que Docker habilita dentro de las redes definidas por el usuario. La aplicación no se conecta a una dirección IP fija ni a localhost, sino al nombre del servicio declarado en el archivo de orquestación, es decir, db. Emplear localhost constituiría un error, puesto que dentro de un contenedor ese nombre apunta al propio contenedor y no a los demás. Emplear una dirección IP tampoco sería adecuado, ya que esta cambia cada vez que el contenedor se recrea, mientras que el nombre del servicio permanece estable.

Adicionalmente, el arranque se encuentra ordenado mediante la directiva depends_on con la condición service_healthy, de modo que el contenedor de la aplicación solo inicia cuando el

motor de base de datos responde efectivamente a la verificación de salud. Esta precaución evita fallos intermitentes durante la ejecución automatizada del pipeline.

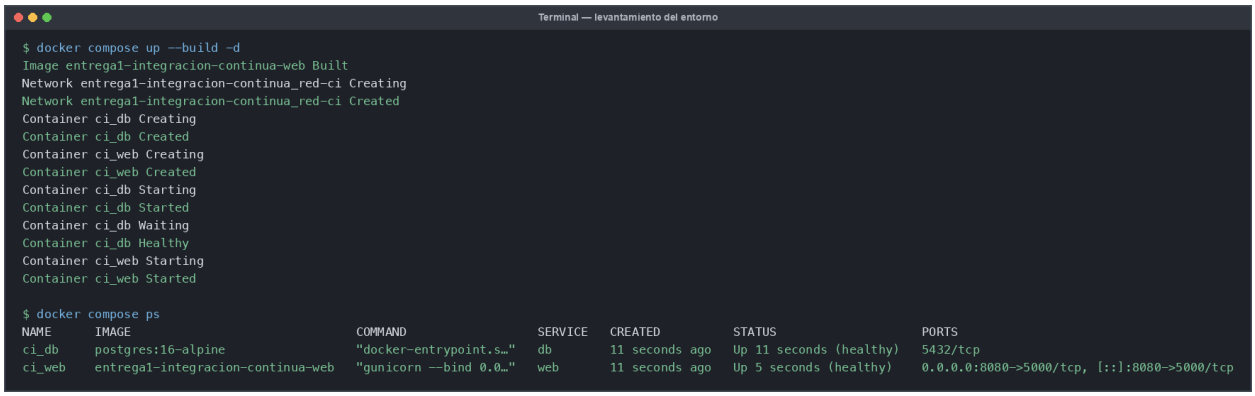
Evidencias De Ejecución

A continuación se presentan las evidencias obtenidas durante la ejecución real del entorno en la máquina local. Las salidas que se muestran corresponden a los comandos efectivamente ejecutados y se reproducen de forma literal.

Evidencias de levantamiento del entorno

Figura 2

Construcción de las imágenes y creación de los contenedores con Docker Compose.



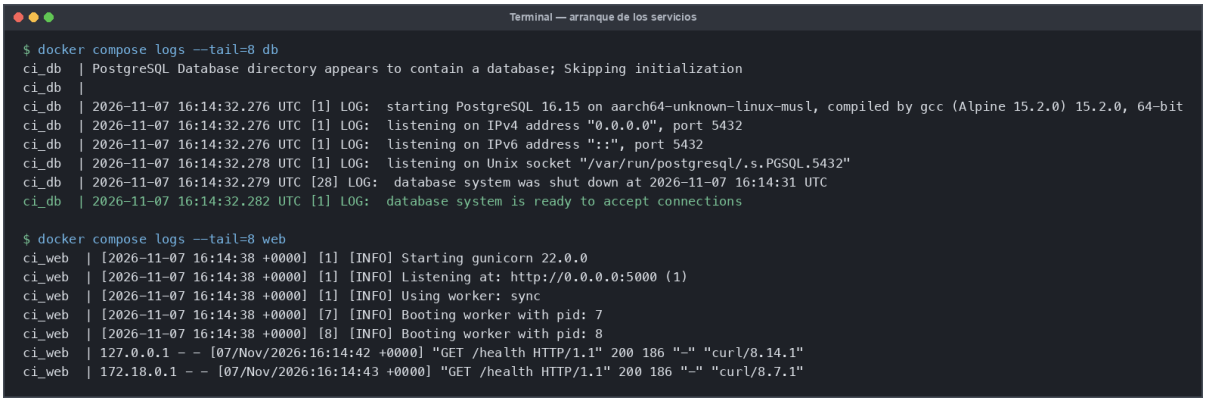
Nota. Salida de consola registrada durante la ejecución del entorno.

Se validó la ejecución de la infraestructura mediante Docker Compose. El comando docker compose up --build -d construyó la imagen del servicio web a partir del Dockerfile del proyecto, creó la red red-ci y levantó los dos contenedores. La consulta posterior con docker compose ps muestra ambos contenedores en estado healthy, evidenciando el correcto levantamiento del entorno de trabajo.

Evidencias de arranque de los servicios

Figura 3

Registros de arranque de los contenedores ci_db y ci_web.



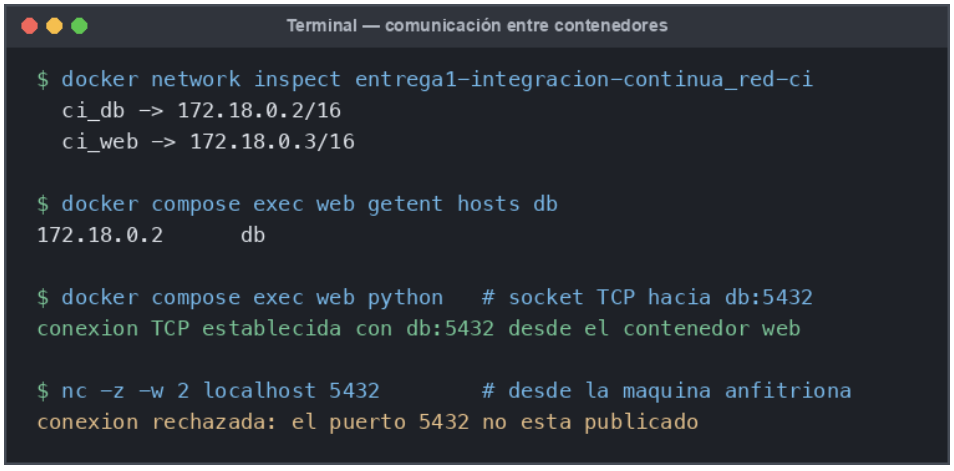
Nota. Salida de consola registrada durante la ejecución del entorno.

Se verificó el arranque correcto de ambos contenedores mediante la inspección de los registros con docker compose logs. El motor PostgreSQL indicó que el sistema de base de datos se encontraba listo para aceptar conexiones, mientras que el servicio web informó que Gunicorn quedó a la escucha en el puerto 5000 y que sus procesos de trabajo iniciaron correctamente.

Comunicación entre los contenedores

Figura 4

Inspección de la red red-ci, resolución de nombres y prueba de conexión.



Nota. Salida de consola registrada durante la ejecución del entorno.

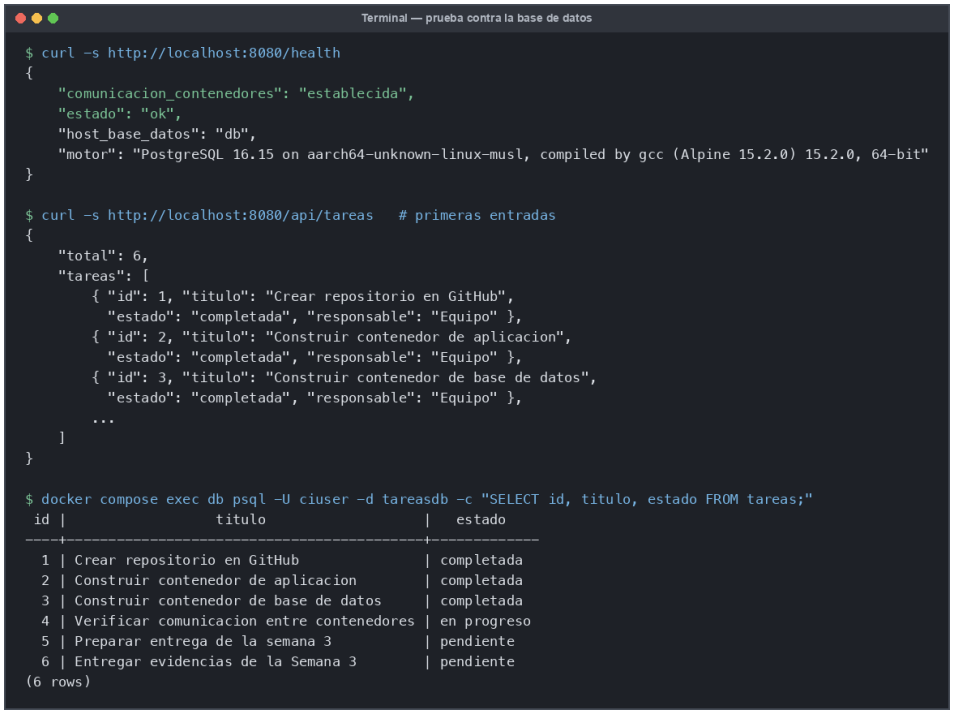
La inspección de la red red-ci muestra que los contenedores ci_db y ci_web se encuentran conectados a la misma red de tipo bridge, con las direcciones 172.18.0.2 y 172.18.0.3 respectivamente. Desde el contenedor de la aplicación, el nombre db se resuelve correctamente a la dirección del contenedor de base de datos y se establece una conexión TCP efectiva hacia el puerto 5432.

Resulta especialmente significativo el último comando de la figura: desde la máquina anfitriona, la conexión al puerto 5432 es rechazada, puesto que el contenedor de base de datos no publica ese puerto. En consecuencia, la única vía de acceso posible es la red interna de Docker, lo cual demuestra que el intercambio de datos ocurre efectivamente entre contenedores y no a través del equipo anfitrión.

Prueba de comunicación con la base de datos

Figura 5

Consulta del estado de la conexión, lectura de datos y verificación en PostgreSQL.



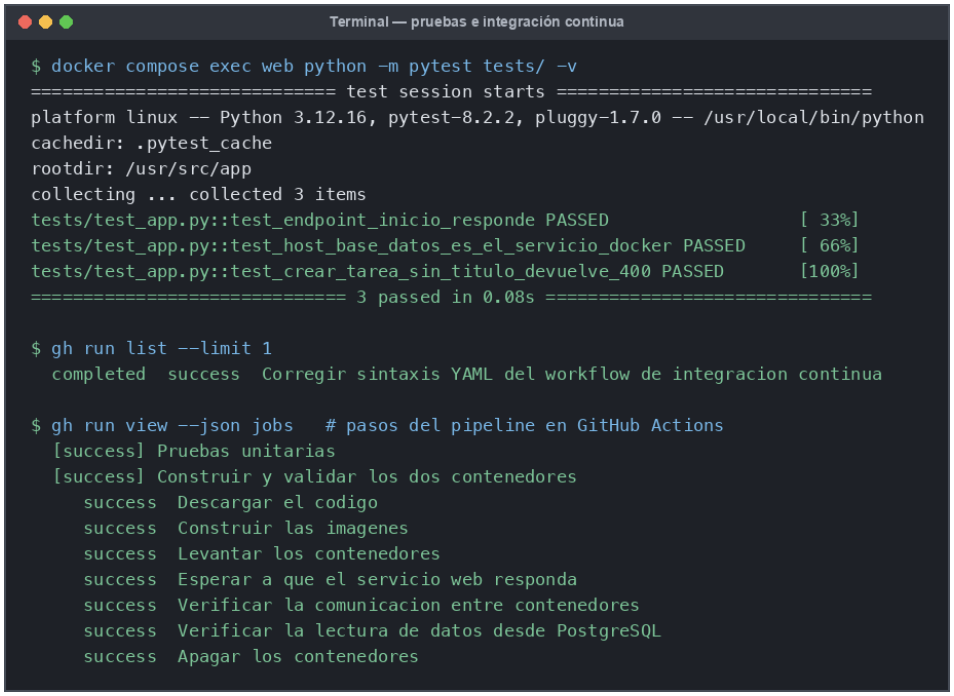
Nota. Salida de consola registrada durante la ejecución del entorno.

El recurso /health solicita al motor de base de datos su versión y devuelve el valor establecida en el campo comunicacion_contenedores, lo que confirma que el servicio web consultó efectivamente a PostgreSQL. El recurso /api/tareas devuelve los registros almacenados, y la consulta ejecutada directamente dentro del contenedor de base de datos mediante psql muestra esos mismos registros. De esta forma queda demostrada la comunicación en ambos sentidos: lectura y escritura.

Evidencias de integración continua

Figura 6

Pruebas unitarias y ejecución del pipeline en GitHub Actions.



Nota. Salida de consola registrada durante la ejecución del entorno.

Como preparación para las entregas siguientes se configuró un pipeline de integración continua en GitHub Actions, el cual se ejecuta en cada push y en cada pull request sobre las ramas main y develop (GitHub, 2026). El pipeline instala las dependencias, ejecuta la suite de pruebas unitarias, construye las imágenes, levanta los dos contenedores en un servidor limpio y verifica que la comunicación entre ellos funcione. La figura muestra las tres pruebas unitarias aprobadas y todos los pasos del pipeline finalizados de forma satisfactoria.

Enlace A Repositorio De GitHub

El código fuente completo del proyecto, junto con los archivos de contenerización, las pruebas y la definición del pipeline, se encuentra versionado y disponible públicamente en la siguiente dirección:

https://github.com/grimmdex/entrega1-integracion-continua

Referencias

Docker Inc. (2026). *Docker documentation*. https://docs.docker.com/

Fowler, M. (2006). *Continuous integration*. https://martinfowler.com/articles/continuousIntegration.html

GitHub. (2026). *GitHub Actions documentation*. https://docs.github.com/actions

Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

Pallets. (2026). *Flask documentation*. https://flask.palletsprojects.com/

PostgreSQL Global Development Group. (2026). *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/16/

Politécnico Grancolombiano. (2026). *Proyecto de software basado en herramientas de integración continua* [Guía del proyecto grupal]. Módulo Énfasis Profesional I.